

Problem Set #4

Due date: October 18, 2024, 11:59 PM

REMINDER: As you work on your coding assignments, it is important to remember that passing the examples provided does not guarantee full credit. While these examples can serve as a helpful starting point, it is ultimately your responsibility to create additional tests, ensure that it is functioning correctly and meets all the requirements and specifications outlined in the assignment instructions. Your grade for this assignment is divided in two parts:

50% > Passing all post-due date tests
(no partial credit for failed cases)

50% > Clarity and design of your code
(Class Coding Standards, requirements/specifications)

Before you start...

- You are not allowed to use material that has not been covered in class.
 - The only string methods you are allowed to use are: `split`, `strip` and `join`
 - The only list methods you are allowed to use are: `append`, `pop` and `del`
 - You can use the `len` function and slicing
- Your code must be written using Python language.
- Function names must be defined as stated in the function description. Failure to comply with this requirement will result in a 0 score for the visible and post-due date testing.
- No code should be written outside functions, including function calls. All testing code must be done inside the `main` function
- Your functions should NOT have any `input()` calls in its body.
- Your code should be concise and efficient without useless code or redundant code.
- Failure to follow the assignment's requirements will result in a zero score for the assignment even if the autograder gives you full score.
- Code submitted with syntax errors does not receive ANY credit.
- Review the Grading Notes provided in the Canvas Assignment that includes the evaluation criteria for Clarity and Design.
- You will be writing several functions, but they will all be saved in one file: `PS4.py`. Please save all the functions in this one file or you will lose points.
- Ask questions using our Problem Set 4 channel in Microsoft Teams.

Part 1: Warm-up

Problem 1.1 (10 pts) Write the Python code to implement the function `count_vowels` that takes in a string as the input and returns the number of vowels in the string. You should also account for case matching (should detect both lowercase and uppercase vowels). Testing if a character is a vowel can be a user-defined function

➖ To receive any credit for this problem, you are not allowed to use any built-in functions or modules to count

Examples:

```
>>> count_vowels("programming")
3
>>> count_vowels("App1E")
2
>>> count_vowels("CMPsc131")
0
```

Problem 1.2 (10 pts) Write the Python code to implement the function `hailstone` that takes a positive integer and returns a list with the hailstone sequence starting at the given number until termination when *num* is 1. The hailstone sequence is generated as follows:

- If *num* is even, then the next number in the sequence is $num/2$.
- If *num* is odd, then the next number in the sequence is $3 * num + 1$.
- Continue this process until $num = 1$

➖ The use of type conversion such as `int()` is not allowed.

Examples:

```
>>> hailstone(5)
[5, 16, 8, 4, 2, 1]
>>> hailstone(6)
[6, 3, 10, 5, 16, 8, 4, 2, 1]
```

Problem 1.3 (10 pts) Write the Python code to implement the function `get_second` that takes a string of words separated by commas as input and returns the second word in the string, with any leading or trailing spaces from the second word removed. You can assume that the string has at least two commas.

Examples:

```
>>> get_second('mouse, cat, dog, pig, lion')
'cat'
>>> get_second('apple,          pear          , banana')
'pear'
```

Part 2: Pig Latin

Problem 2.1 (10 pts) Write the Python code to implement the function `find` that takes a string of size 1, *char*, a string, *text*, and a positive integer *start* as input. The function returns the index of the first occurrence of *char* in *text* starting from *start*. If *char* is not found or the string *text* is empty, return -1.

➡ To receive any credit for this problem, you are not allowed to use the built-in functions `find()` or `index()`

Examples:

```
>>> find('p', 'hello', 0)
-1
>>> find('e', 'hello', 2)
-1
>>> find('e', 'hello', 0)
1
>>> find('l', 'hello', 1)
2
>>> find('l', 'hello', 3)
3
```

Problem 2.2 (10 pts) Write the Python code to implement the function `find_first_vowel` that takes a non-empty string of lowercase letters as input. The function returns the position of the first vowel, or the size of the string if no vowels are present in it. The vowels are 'a', 'e', 'i', 'o', 'u', as well as any 'y' that is not the first letter of the input string. Your implementation must use the `find` function from Problem 2.1

➡ To receive any credit for this problem, you are not allowed to use the built-in functions `find()` or `index()`

Examples:

```
>>> find_first_vowel('hello')
1
>>> find_first_vowel('drone')
2
>>> find_first_vowel('r2d2')
4
>>> find_first_vowel('yummy')
1
>>> find_first_vowel('dry')
2
```

Problem 2.3 (10 pts) Pig Latin is a language game in which words in English are altered, usually by adding a fabricated suffix or by moving the onset or initial consonant or consonant cluster of a word to the end of the word and adding a vocalic syllable to create such a suffix. For this problem, we will convert words into Pig Latin according to the following rules:

1. The vowels are 'a', 'e', 'i', 'o', 'u', as well as any 'y' that is not the first letter of a word. All other letters are consonants.
2. If the word begins with a vowel, we append 'hay' to the end of the word to get the Pig Latin equivalent.
3. If the word starts with 'q', we will always assume it is followed by 'u', so we will move 'qu' to the end of the word, and append 'ay'.

4. If the word begins with a consonant, we move all the consonants up to the first vowel (if any) to the end and add 'ay'.

Write the Python code to implement the function `get_pig_latin` that takes a non-empty string of lowercase letters as input. The function returns a string that represents the input string in Pig Latin form. Your implementation must use the `find_first_vowel` function from Problem 2.2.

➡ To receive any credit for this problem, you are not allowed to use the built-in functions `find()` or `index()`

Examples:

```
>>> get_pig_latin('ortogonal')
'ortogonahay'
>>> get_pig_latin('quacking')
'ackingquay'
>>> get_pig_latin('blue')
'ueblay'
>>> get_pig_latin('dry')
'ydray'
>>> get_pig_latin('kmr')
'kmray'
```

Part 3: We love lists!

Problem 3.1 (10 pts) Write a Python code to implement the function `longest_word` that takes in a sentence (string of words separated by spaces) as an input and returns the longest word from the sentence. If a sentence has multiple longest words, return the first one that appears

Examples:

```
>>> sentence = "I love programming in Python"
>>> longest_word(sentence)
'programming '

>>> sentence = "The quick brown fox jumps over the lazy dog"
>>> longest_word(sentence)
'quick'
```

Problem 3.2 (10 pts) Write a Python code to implement the function `replace_vowels_in_list` that takes a list of strings and mutates the list by replacing all vowels in each word with the letter 'z'. The function does not return a value

➡ To receive any credit for this problem, you are not allowed to use the built-in `replace()` or any other in-built functions from the standard library.

Examples:

```
>>> my_list = ['hello', 'world']
>>> replace_vowels_in_list(my_list)
>>> my_list
['hzllz', 'wzrld']
```

Problem 3.3 (10 pts) Write the Python code to implement the function `alternating_sum` that takes a list of numbers. The function returns a float value that represents the alternating sum (where the sign alternates from positive to negative or vice versa) of all elements in the list. When the list is empty, it returns 0.0.

Examples:

```
>>> alternating_sum([1, 5])          # 1 - 5
-4.0
>>> alternating_sum([1, 5, 17])      # 1 - 5 + 17
13.0
>>> alternating_sum([1, 5, 17, 4])   # 1 - 5 + 17 - 4
9.0
```

Problem 3.4 (10 pts) Let us define a sliding window as a part of a list of size w . Write the function `max_in_window` that takes a list of numbers and a positive integer as input. The function returns a list that contains the maximum value of each of the sliding windows without destroying the original list.

⊖ To receive any credit for this problem, you are not allowed to use the built-in function `max()`, you must define your own standalone function.

Examples:

```
>>> max_in_window([2, 5, 12, 3, 4], 2) # windows are [2, 5], [5, 12], [12, 3], [3, 4]
[5, 12, 12, 4]
>>> max_in_window([1, 3, 5, 1, 2, 4, 7, 8], 4)
[5, 5, 5, 7, 8]
>>> max_in_window([1, 3, 0, 3, 5, 3, 6, 2, 8], 3)
[3, 3, 5, 5, 6, 6, 8]
```